SPRING BOOT

Spring — Bookstore Chapter 4

Пошаговое учебное руководство
от условия до проверки и сдачи

Обязательная работа

Учебный комплект • 2026

> Статус: **обязательная** работа. Это самостоятельный учебный срез Chapter 4 поверх Bookstore Chapter 3. MySQL сюда не входит: Optional разобран отдельно в `09-spring-bookstore-mysql.md`.

Что требуется

- Изучить Student–Department из Moodle как пример связи; отдельный StudentList сдавать не требуется.
- Создать JPA entity `Category` с `id`, `name`, конструкторами, getters и setters.
- Реализовать двустороннюю связь: `Book.category` — many-to-one, `Category.books` — one-to-many.
- Создать `CategoryRepository extends CrudRepository<Category, Long>`.
- Через `CommandLineRunner` сохранить несколько категорий и связанные книги.
- Передать категории из controller в `Model` add/edit формы.
- Добавить dropdown, показывающий имена категорий и отправляющий `categoryId`.
- При save найти Category по `categoryId` и связать её с книгой.
- Показать category name каждой книги в `/booklist`.
- Проверить связь repository-тестом, web-поведение MockMvc-тестами и выполнить Maven gate на H2.
- Отправить изменения в собственный GitHub-репозиторий Bookstore.

Границы результата

Entity, обе стороны связи, repository, seed, dropdown и вывод имени категории — обязательный Chapter 4. Отдельный CRUD категорий не требуется. DTO `BookForm`, защита от подмены id, понятные 404, повторно безопасный initializer, Bootstrap и расширенные тесты — улучшения accumulated-проекта. MySQL/XAMPP — только Optional 09 и не нужен ни обычному запуску, ни тестам Chapter 4.

Что получится

H2 создаст `CATEGORY` и `BOOK`; внешний ключ `BOOK.CATEGORY_ID` хранит связь. В форме появятся Programming, Architecture и Databases. Контроллер превратит отправленный id в managed Category, установит owning side через `book.setCategory`, а список выведет `book.category.name`.

Подготовка на macOS

Нужны Git и JDK 17+. Maven отдельно не нужен: используется Maven Wrapper.

macOS:

КОД · BASH

```
xcode-select --install              # только если Git ещё не установлен
java -version
git --version
cd fullstack-homework-kit/projects/spring-boot/bookstore
chmod +x mvnw
./mvnw --version
```

Windows PowerShell

КОД · POWERSHELL

```
java -version
git --version
Set-Location fullstack-homework-kit\projects\spring-boot\bookstore
.\mvnw.cmd --version
```

Если JDK отсутствует, установите Temurin 17/21 или другой JDK, перезапустите терминал и повторите проверку.

Создание с нуля

Chapter 4 продолжает работающий Chapter 3. Для полностью чистого воспроизведения создайте в Spring Initializr проект Spring Boot 3.x, Java 17, Maven с Spring Web, Thymeleaf, Spring Data JPA, H2 и Spring Boot Test, затем замените ручные файлы листингами ниже. `mvnw`, `mvnw.cmd` и `.mvn/wrapper/**` генерирует Initializr — вручную их не переписывают.

До production-кода создайте `CategoryRepositoryTest`, запустите его и увидьте RED из-за отсутствующих Category/repository/relationship. После каждого минимального изменения повторяйте тест.

Структура файлов

КОД · TEXT

```
src/main/java/com/example/bookstore/
├── BookstoreApplication.java
├── Book.java
├── Category.java
├── BookForm.java
├── BookRepository.java
├── CategoryRepository.java
├── BookstoreDataInitializer.java
└── BookController.java
src/main/resources/
├── application.properties
├── templates/addbook.html
├── templates/booklist.html
└── static/css/{bootstrap.min.css,bootstrap-LICENSE.txt}
src/test/java/com/example/bookstore/
├── BookstoreApplicationTests.java
├── BookRepositoryTest.java
├── CategoryRepositoryTest.java
├── BookstoreDataInitializerTest.java
└── BookControllerTest.java
```

Пошаговая реализация

1. pom.xml

Ниже — фактический cumulative `pom.xml`, чтобы листинг совпадал с готовым проектом. Блок Connector/J отмечен как накопленное Optional 09: при воспроизведении строго чистого Chapter 4 его можно не добавлять, остальные строки остаются теми же.

КОД · XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.16</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>bookstore</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>bookstore</name>
    <description>Учебное приложение Spring Boot Bookstore</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

```
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

`starter-web` даёт MVC/сервер, `starter-thymeleaf` — templates, `starter-data-jpa` — JPA/Hibernate, H2 — runtime database, `starter-test` — JUnit/AssertJ/MockMvc. Plugin собирает запускаемый jar. Connector/J присутствует в фактическом cumulative-проекте после Optional 09, но не активирует MySQL без profile и не является требованием чистого Chapter 4.

2. BookstoreApplication.java

КОД · JAVA

```
package com.example.bookstore;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BookstoreApplication {
    public static void main(String[] args) {
        SpringApplication.run(BookstoreApplication.class, args);
    }
}
```

`@SpringBootApplication` включает configuration, auto-configuration и component scan. `main` создаёт Spring context, datasource, repositories, controller и сервер. Файл обычно уже есть в Chapter 3.

3. Book.java

КОД · JAVA

```
package com.example.bookstore;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;

@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String author;
    private String isbn;
    private int publicationYear;

    @ManyToOne
    @JoinColumn(name = "category_id")
    private Category category;

    public Book() {
```

```
    }

    public Book(Long id, String title, String author, String isbn, int publicationYear) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.isbn = isbn;
        this.publicationYear = publicationYear;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }

    public int getPublicationYear() {
        return publicationYear;
    }

    public void setPublicationYear(int publicationYear) {
        this.publicationYear = publicationYear;
    }

    public Category getCategory() {
        return category;
    }

    public void setCategory(Category category) {
        if (this.category == category) {
            return;
        }

        Category previousCategory = this.category;
        this.category = category;
        if (previousCategory != null) {
            previousCategory.getBooks().remove(this);
        }
        if (category != null && !category.getBooks().contains(this)) {
            category.getBooks().add(this);
        }
```

```
    }
}
```

`@Entity` делает класс persistent. `@Id`/`@GeneratedValue(IDENTITY)` делегируют id базе. Открытый no-arg constructor нужен JPA; второй удобен seed/tests. `@ManyToOne` и `@JoinColumn` делают `Book.category` владельцем: именно `BOOK.CATEGORY_ID` меняется при save. `setCategory` ничего не делает при повторной установке того же объекта, удаляет книгу из коллекции прежней категории, меняет owning side и добавляет книгу в новую коллекцию. Так обе Java-стороны остаются согласованными и при первом назначении, и при переносе книги; это не включает cascade. Getters/setters нужны JPA, MVC и Thymeleaf.

4. Category.java

КОД · JAVA

```
package com.example.bookstore;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import java.util.ArrayList;
import java.util.List;

@Entity
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "category")
    private List<Book> books = new ArrayList<>();

    public Category() {
    }

    public Category(Long id, String name) {
        this.id = id;
        this.name = name;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public List<Book> getBooks() {
        return books;
    }

    public void setBooks(List<Book> books) {
```

```
        this.books = books;
    }
}
```

`Category` — новая entity. `mappedBy = "category"` точно называет поле owner в `Book`; эта inverse side не создаёт второй FK. Коллекция сразу инициализирована, поэтому getter не возвращает null. Setter нужен по условию. Cascade/orphan removal намеренно отсутствуют: удаление category вместе с books не задано.

5. BookRepository.java

КОД · JAVA

```
package com.example.bookstore;

import org.springframework.data.repository.CrudRepository;

public interface BookRepository extends CrudRepository<Book, Long> {
}
```

Spring Data создаёт реализацию во время старта. Generic-параметры — entity и тип id; наследуются `findAll`, `findById`, `save`, `saveAll`, `count`, `delete`. Файл не меняется относительно Chapter 3, но приведён для самостоятельной сборки.

6. CategoryRepository.java

КОД · JAVA

```
package com.example.bookstore;

import org.springframework.data.repository.CrudRepository;

public interface CategoryRepository extends CrudRepository<Category, Long> {
}
```

Это новый обязательный repository. `findAll` заполняет dropdown, `findById` разрешает `categoryId`, `save` создаёт seed, `count` защищает initializer от дублей.

7. BookForm.java

КОД · JAVA

```
package com.example.bookstore;

public class BookForm {
    private String title;
    private String author;
    private String isbn;
    private int publicationYear;

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
```

```
        this.author = author;
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }

    public int getPublicationYear() {
        return publicationYear;
    }

    public void setPublicationYear(int publicationYear) {
        this.publicationYear = publicationYear;
    }
}
```

Spring MVC создаёт DTO неявным no-arg constructor и вызывает setters по именам inputs. Здесь нет id/category: клиент не выбирает entity id, а `categoryId` проверяется отдельно. DTO — accumulated-защита; исходное условие её явно не требует.

8. BookstoreDataInitializer.java

КОД · JAVA

```
package com.example.bookstore;

import java.util.List;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BookstoreDataInitializer {
    @Bean
    CommandLineRunner initialData(CategoryRepository categoryRepository, BookRepository
bookRepository) {
        return arguments -> {
            if (categoryRepository.count() > 0 || bookRepository.count() > 0) {
                return;
            }

            Category programming = categoryRepository.save(new Category(null, "Programming"));
            Category architecture = categoryRepository.save(new Category(null, "Architecture"));
            Category databases = categoryRepository.save(new Category(null, "Databases"));

            Book cleanCode = new Book(null, "Clean Code", "Robert C. Martin", "9780132350884",
2008);
            cleanCode.setCategory(architecture);
            Book effectiveJava = new Book(null, "Effective Java", "Joshua Bloch", "9780134685991",
2018);
            effectiveJava.setCategory(programming);
            Book springInAction = new Book(null, "Spring in Action", "Craig Walls",
"9781617297571", 2022);
            springInAction.setCategory(programming);

            bookRepository.saveAll(List.of(cleanCode, effectiveJava, springInAction));
        };
    }
}
```

`@Configuration` и `@Bean` регистрируют `CommandLineRunner`; repositories внедряются через параметры метода. Категории сохраняются первыми и становятся managed, затем owner каждой книги получает category и `saveAll` пишет FK. Cascade не нужен. Проверка counts — accumulated-улучшение для постоянной MySQL, чтобы повторный старт не дублировал seed.

9. BookController.java

КОД · JAVA

```
package com.example.bookstore;

import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.server.ResponseStatusException;

@Controller
public class BookController {
    private final BookRepository bookRepository;
    private final CategoryRepository categoryRepository;

    public BookController(BookRepository bookRepository, CategoryRepository categoryRepository) {
        this.bookRepository = bookRepository;
        this.categoryRepository = categoryRepository;
    }

    @GetMapping("/booklist")
    public String bookList(Model model) {
        model.addAttribute("books", bookRepository.findAll());
        return "booklist";
    }

    @GetMapping("/add")
    public String addBookForm(Model model) {
        addFormModel(model, new Book());
        return "addbook";
    }

    @PostMapping("/save")
    public String addBook(@ModelAttribute BookForm form, @RequestParam Long categoryId) {
        return saveBook(new Book(), form, categoryId);
    }

    @PostMapping("/save/{id}")
    public String updateBook(@PathVariable Long id, @ModelAttribute BookForm form, @RequestParam
Long categoryId) {
        return saveBook(requiredBook(id), form, categoryId);
    }

    private String saveBook(Book book, BookForm form, Long categoryId) {
        Category category = categoryRepository.findById(categoryId)
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Category not
found"));
        book.setTitle(form.getTitle());
        book.setAuthor(form.getAuthor());
        book.setIsbn(form.getIsbn());
        book.setPublicationYear(form.getPublicationYear());
        book.setCategory(category);
        bookRepository.save(book);
        return "redirect:/booklist";
```

```
    }

    @GetMapping("/edit/{id}")
    public String editBookForm(@PathVariable Long id, Model model) {
        addFormModel(model, requiredBook(id));
        return "addbook";
    }

    @GetMapping("/delete/{id}")
    public String deleteBook(@PathVariable Long id) {
        bookRepository.delete(requiredBook(id));
        return "redirect:/booklist";
    }

    private Book requiredBook(Long id) {
        return bookRepository.findById(id)
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Book not
found"));
    }

    private void addFormModel(Model model, Book book) {
        model.addAttribute("book", book);
        model.addAttribute("categories", categoryRepository.findAll());
    }
}
```

`@Controller` регистрирует MVC handler. Constructor injection гарантирует оба repositories. `addFormModel` — центральный шаг Chapter 4: и add, и edit получают `book` и `categories`. `@ModelAttribute` связывает текстовые поля с DTO, `@RequestParam categoryId` — dropdown. `saveBook` находит managed Category, копирует разрешённые поля, устанавливает owner и сохраняет. `requiredBook`, отдельные POST add/edit и 404 — accumulated-защита от подмены id.

10. application.properties

КОД · PROPERTIES

```
spring.application.name=bookstore
spring.datasource.url=jdbc:h2:mem:bookstore;MODE=MySQL;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Это default H2 profile. `create-drop` строит схему из entities; SQL logging показывает FK; console работает по `/h2-console`. H2 `MODE=MySQL` полезен для совместимости, но не заменяет реальный MySQL. Пустой пароль относится только к локальной in-memory H2, не к Optional MySQL.

11. templates/addbook.html

КОД · HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="https://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title th:text="${book.id == null ? 'Add Book' : 'Edit Book'}">Add Book</title>
```

```
    <link type="text/css" rel="stylesheet" href="/css/bootstrap.min.css"
th:href="@{/css/bootstrap.min.css}">
</head>
<body>
<main class="container py-4">
    <h1 th:text="${book.id == null ? 'Add Book' : 'Edit Book'}">Add Book</h1>
    <form th:action="${book.id == null} ? @{/save} : @{/save/{id}(id=${book.id})}"
          th:object="${book}" method="post" class="mt-4">
        <div class="mb-3">
            <label class="form-label" for="title">Title</label>
            <input class="form-control" id="title" type="text" th:field="*{title}" required>
        </div>
        <div class="mb-3">
            <label class="form-label" for="author">Author</label>
            <input class="form-control" id="author" type="text" th:field="*{author}" required>
        </div>
        <div class="mb-3">
            <label class="form-label" for="isbn">ISBN</label>
            <input class="form-control" id="isbn" type="text" th:field="*{isbn}" required>
        </div>
        <div class="mb-3">
            <label class="form-label" for="publicationYear">Publication year</label>
            <input class="form-control" id="publicationYear" type="number"
th:field="*{publicationYear}" required>
        </div>
        <div class="mb-3">
            <label class="form-label" for="categoryId">Category</label>
            <select class="form-select" id="categoryId" name="categoryId" required>
                <option th:each="category : ${categories}"
                        th:value="${category.id}"
                        th:text="${category.name}"
                        th:selected="${book.category != null and book.category.id ==
category.id}"></option>
            </select>
        </div>
        <button class="btn btn-success" type="submit">Save</button>
        <a class="btn btn-secondary" th:href="@{/booklist}">Cancel</a>
    </form>
</main>
</body>
</html>
```

`th:object` задаёт объект для `th:field`; тернарный `th:action` различает add/edit. Dropdown итерирует `${categories}`: `th:text` показывает name, `th:value` отправляет id. `name="categoryId"` обязан совпасть с `@RequestParam`. `th:selected` сохраняет выбор на edit и сначала безопасно проверяет category на null. `required` — только browser validation; server всё равно проверяет id.

12. templates/booklist.html

КОД · HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="https://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bookstore</title>
    <link type="text/css" rel="stylesheet" href="/css/bootstrap.min.css"
th:href="@{/css/bootstrap.min.css}">
</head>
<body>
<main class="container py-4">
    <h1 class="mb-4">Bookstore</h1>
    <p><a class="btn btn-success" th:href="@{/add}">Add Book</a></p>
    <table class="table table-striped">
```

```
        <thead>
        <tr>
            <th scope="col">ID</th>
            <th scope="col">Title</th>
            <th scope="col">Author</th>
            <th scope="col">ISBN</th>
            <th scope="col">Publication year</th>
            <th scope="col">Category</th>
            <th scope="col">Actions</th>
        </tr>
        </thead>
        <tbody>
        <tr th:each="book : ${books}">
            <td th:text="${book.id}"></td>
            <td th:text="${book.title}"></td>
            <td th:text="${book.author}"></td>
            <td th:text="${book.isbn}"></td>
            <td th:text="${book.publicationYear}"></td>
            <td th:text="${book.category != null ? book.category.name : ''}"></td>
            <td>
                <a class="btn btn-danger" th:href="@{/delete/{id}(id=${book.id})}">Delete</a>
                <a class="btn btn-outline-secondary"
th:href="@{/edit/{id}(id=${book.id})}">Edit</a>
            </td>
        </tr>
        </tbody>
    </table>
</main>
</body>
</html>
```

`th:each` строит строку из model attribute `books`. Новый обязательный столбец выводит `book.category.name`; тернарное выражение безопасно для старых записей без category. `th:text` экранирует вывод. Edit/delete передают id как path variable.

13. CategoryRepositoryTest.java

Сначала создайте этот тест и увидьте RED на Chapter 3.

КОД · JAVA

```
package com.example.bookstore;

import static org.assertj.core.api.Assertions.assertThat;

import java.lang.reflect.Method;
import java.util.Arrays;
import java.util.List;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;

@DataJpaTest
class CategoryRepositoryTest {
    @Autowired
    private CategoryRepository categoryRepository;

    @Autowired
    private BookRepository bookRepository;

    @Autowired
    private TestEntityManager entityManager;

    @Test
```

```
    void bookBelongsToCategoryAndCategoryLoadsBooks() {
        Category category = categoryRepository.save(new Category(null, "Programming"));
        Book book = new Book(null, "Effective Java", "Joshua Bloch", "9780134685991", 2018);
        book.setCategory(category);
        bookRepository.save(book);

        entityManager.flush();
        entityManager.clear();

        Category reloaded = categoryRepository.findById(category.getId()).orElseThrow();

        assertThat(reloaded.getName()).isEqualTo("Programming");
        assertThat(reloaded.getBooks()).extracting(Book::getTitle)
                .containsExactly("Effective Java");
    }

    @Test
    void movingBookBetweenCategoriesKeepsBothSidesConsistent() {
        Category original = new Category(null, "Programming");
        Category replacement = new Category(null, "Databases");
        Book book = new Book(null, "Effective Java", "Joshua Bloch", "9780134685991", 2018);

        book.setCategory(original);
        book.setCategory(replacement);

        assertThat(book.getCategory()).isSameAs(replacement);
        assertThat(original.getBooks()).doesNotContain(book);
        assertThat(replacement.getBooks()).containsExactly(book);
    }

    @Test
    void booksAttributeCanBeReplacedThroughItsRequiredSetter() throws Exception {
        Category category = new Category(null, "Testing");
        List<Book> replacement = List.of(
                new Book(null, "Test-Driven Development", "Kent Beck", "9780321146533", 2002));
        Method setter = Arrays.stream(Category.class.getMethods())
                .filter(method -> method.getName().equals("setBooks"))
                .filter(method -> Arrays.equals(method.getParameterTypes(), new
Class<?>[]{List.class}))
                .findFirst()
                .orElse(null);

        assertThat(setter).as("Category must expose setBooks(List<Book>)").isNotNull();
        setter.invoke(category, replacement);

        assertThat(category.getBooks()).isSameAs(replacement);
    }
}
```

`@DataJpaTest` поднимает JPA slice на H2 и откатывает транзакцию. `TestEntityManager.flush()` действительно отправляет изменения в БД, а `clear()` удаляет managed-объекты из persistence context; поэтому последующий `findById` загружает связь заново и реально проверяет `mappedBy`, а не возвращает тот же объект из кэша первого уровня. Отдельный тест проверяет перенос книги и обе стороны Java-связи. Reflection-тест проверяет буквальное учебное требование `setBooks(List<Book>)`.

14. BookControllerTest.java — фактический cumulative-файл

КОД · JAVA

```
package com.example.bookstore;

import static org.hamcrest.Matchers.containsString;
import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
```

```
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.redirectedUrl;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.MvcResult;
import org.springframework.web.server.ResponseStatusException;

@SpringBootTest
@AutoConfigureMockMvc
class BookControllerTest {
    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private BookRepository bookRepository;

    @Autowired
    private CategoryRepository categoryRepository;

    @Test
    void bookListRendersSeededBooksAndBootstrapClasses() throws Exception {
        mockMvc.perform(get("/booklist"))
                .andExpect(status().isOk())
                .andExpect(content().contentTypeCompatibleWith(MediaType.TEXT_HTML))
                .andExpect(content().string(containsString("Clean Code")))
                .andExpect(content().string(containsString("class=\"table table-striped\"")))
                .andExpect(content().string(containsString("class=\"btn btn-success\"")))
                .andExpect(content().string(containsString("href=\"/css/bootstrap.min.css\"")));
    }

    @Test
    void editLinkAppearsAfterDeleteLinkAsRequiredByTheExercise() throws Exception {
        MvcResult result = mockMvc.perform(get("/booklist"))
                .andExpect(status().isOk())
                .andReturn();

        String html = result.getResponse().getContentAsString();
        assertThat(html.indexOf(">Delete</a>"))
                .isGreaterThanOrEqualTo(0)
                .isLessThan(html.indexOf(">Edit</a>"));
    }

    @Test
    void bootstrapIsServedLocally() throws Exception {
        mockMvc.perform(get("/css/bootstrap.min.css"))
                .andExpect(status().isOk())
                .andExpect(content().string(containsString("Bootstrap  v5.3.3")));
    }

    @Test
    void addFormRendersTheNamedChapterThreeTemplateWithBookFields() throws Exception {
        mockMvc.perform(get("/add"))
                .andExpect(status().isOk())
                .andExpect(content().string(containsString("Add Book")))
                .andExpect(content().string(containsString("name=\"title\"")))
                .andExpect(content().string(containsString("name=\"author\"")))
                .andExpect(content().string(containsString("name=\"isbn\"")))
                .andExpect(content().string(containsString("name=\"publicationYear\"")));
    }
```

```
    @Test
    void saveRedirectsAndNewBookAppearsInList() throws Exception {
        Category category = categoryRepository.findAll().iterator().next();

        mockMvc.perform(post("/save")
                        .param("title", "Domain-Driven Design")
                        .param("author", "Eric Evans")
                        .param("isbn", "9780321125217")
                        .param("publicationYear", "2003")
                        .param("categoryId", category.getId().toString()))
                .andExpect(status().is3xxRedirection())
                .andExpect(redirectedUrl("/booklist"));

        mockMvc.perform(get("/booklist"))
                .andExpect(content().string(containsString("Domain-Driven Design")));
    }

    @Test
    void addIgnoresAnInjectedBookIdInsteadOfOverwritingAnotherBook() throws Exception {
        Category category = categoryRepository.findAll().iterator().next();
        Book protectedBook = bookRepository.save(new Book(
                null, "Protected title", "Original author", "protected-isbn", 1999));

        mockMvc.perform(post("/save")
                        .param("id", protectedBook.getId().toString())
                        .param("title", "New independent book")
                        .param("author", "New author")
                        .param("isbn", "new-isbn")
                        .param("publicationYear", "2026")
                        .param("categoryId", category.getId().toString()))
                .andExpect(status().is3xxRedirection())
                .andExpect(redirectedUrl("/booklist"));

        Book unchangedBook = bookRepository.findById(protectedBook.getId()).orElseThrow();
        assertThat(unchangedBook.getTitle()).isEqualTo("Protected title");
        assertThat(bookRepository.findAll())
                .extracting(Book::getTitle)
                .contains("New independent book");
    }

    @Test
    void postEditUpdatesOnlyTheRequestedExistingBook() throws Exception {
        Category category = categoryRepository.findAll().iterator().next();
        Book book = bookRepository.save(new Book(
                null, "Before edit", "Before author", "before-isbn", 2001));

        mockMvc.perform(post("/save/{id}", book.getId())
                        .param("title", "After edit")
                        .param("author", "After author")
                        .param("isbn", "after-isbn")
                        .param("publicationYear", "2025")
                        .param("categoryId", category.getId().toString()))
                .andExpect(status().is3xxRedirection())
                .andExpect(redirectedUrl("/booklist"));

        Book updatedBook = bookRepository.findById(book.getId()).orElseThrow();
        assertThat(updatedBook.getTitle()).isEqualTo("After edit");
        assertThat(updatedBook.getAuthor()).isEqualTo("After author");
        assertThat(updatedBook.getIsbn()).isEqualTo("after-isbn");
        assertThat(updatedBook.getPublicationYear()).isEqualTo(2025);
        assertThat(updatedBook.getCategory().getId()).isEqualTo(category.getId());
    }

    @Test
    void postEditForUnknownBookReturnsTheControllerNotFoundError() throws Exception {
        Category category = categoryRepository.findAll().iterator().next();
```

```
        mockMvc.perform(post("/save/999999")
                        .param("title", "Missing")
                        .param("author", "Nobody")
                        .param("isbn", "missing-isbn")
                        .param("publicationYear", "2026")
                        .param("categoryId", category.getId().toString()))
                .andExpect(status().isNotFound())
                .andExpect(result -> assertThat(result.getResolvedException())
                        .isInstanceOf(ResponseStatusException.class)
                        .hasMessageContaining("Book not found"));
    }

    @Test
    void editLoadsExistingBookAndDeleteRemovesIt() throws Exception {
        Book book = bookRepository.save(new Book(
                null, "Working Effectively with Legacy Code", "Michael Feathers", "9780131177055",
2004));

        mockMvc.perform(get("/edit/{id}", book.getId()))
                .andExpect(status().isOk())
                .andExpect(content().string(containsString("Working Effectively with Legacy
Code")));

        mockMvc.perform(get("/delete/{id}", book.getId()))
                .andExpect(status().is3xxRedirection())
                .andExpect(redirectedUrl("/booklist"));

        assertThat(bookRepository.findById(book.getId())).isEmpty();
    }

    @Test
    void unknownBookIdsReturnNotFound() throws Exception {
        mockMvc.perform(get("/edit/999999"))
                .andExpect(status().isNotFound());
        mockMvc.perform(get("/delete/999999"))
                .andExpect(status().isNotFound());
    }

    @Test
    void addFormListsCategoryNames() throws Exception {
        mockMvc.perform(get("/add"))
                .andExpect(status().isOk())
                .andExpect(content().string(containsString("name=\"categoryId\"")))
                .andExpect(content().string(containsString(">Programming<")))
                .andExpect(content().string(containsString(">Architecture<")));
    }

    @Test
    void savedBookShowsSelectedCategory() throws Exception {
        Category category = categoryRepository.findAll().iterator().next();

        mockMvc.perform(post("/save")
                        .param("title", "Refactoring")
                        .param("author", "Martin Fowler")
                        .param("isbn", "9780134757599")
                        .param("publicationYear", "2018")
                        .param("categoryId", category.getId().toString()))
                .andExpect(status().is3xxRedirection());

        mockMvc.perform(get("/booklist"))
                .andExpect(content().string(containsString("Refactoring")))
                .andExpect(content().string(containsString(category.getName())));
    }

    @Test
    void unknownCategoryReturnsNotFound() throws Exception {
        mockMvc.perform(post("/save")
```

```
                        .param("title", "Rejected")
                        .param("author", "Nobody")
                        .param("isbn", "000")
                        .param("publicationYear", "2026")
                        .param("categoryId", "999999"))
                .andExpect(status().isNotFound());
    }
}
```

Этот листинг дословно отражает cumulative-проект. `@SpringBootTest` поднимает полный контекст и seed; `@AutoConfigureMockMvc` проверяет HTTP без внешнего сервера. Последние три теста непосредственно защищают Chapter 4: model/dropdown contract, цепочку request → repository → relationship → view и 404 неизвестной category. Остальные — регрессии Chapter 3, edit/delete, DTO-защита и локальный Bootstrap; они полезны, но не расширяют исходные требования 04.

15. BookRepositoryTest.java

КОД · JAVA

```
package com.example.bookstore;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;

@DataJpaTest
class BookRepositoryTest {
    @Autowired
    private BookRepository bookRepository;

    @Test
    void savesAndReadsEveryBookField() {
        Book saved = bookRepository.save(new Book(
                null, "Clean Code", "Robert C. Martin", "9780132350884", 2008));

        Book reloaded = bookRepository.findById(saved.getId()).orElseThrow();

        assertThat(reloaded.getTitle()).isEqualTo("Clean Code");
        assertThat(reloaded.getAuthor()).isEqualTo("Robert C. Martin");
        assertThat(reloaded.getIsbn()).isEqualTo("9780132350884");
        assertThat(reloaded.getPublicationYear()).isEqualTo(2008);
    }
}
```

Это неизменённая регрессия Chapter 3 для базовых полей Book. Relationship вынесен в CategoryRepositoryTest, поэтому причины падения не смешиваются.

16. BookstoreDataInitializerTest.java

КОД · JAVA

```
package com.example.bookstore;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.test.context.SpringBootTest;
```

```
@SpringBootTest
class BookstoreDataInitializerTest {
    @Autowired
    private BookRepository bookRepository;

    @Autowired
    private CategoryRepository categoryRepository;

    @Autowired
    @Qualifier("initialData")
    private CommandLineRunner initialData;

    @Test
    void runningInitializerAgainDoesNotDuplicateBooksOrCategories() throws Exception {
        long booksBefore = bookRepository.count();
        long categoriesBefore = categoryRepository.count();

        initialData.run();

        assertThat(bookRepository.count()).isEqualTo(booksBefore);
        assertThat(categoryRepository.count()).isEqualTo(categoriesBefore);
    }
}
```

Это тест accumulated-улучшения. `@Qualifier("initialData")` выбирает bean по имени метода. Повторный вызов runner не меняет counts — особенно важно после перехода к постоянной MySQL.

17. BookstoreApplicationTests.java — фактический cumulative-файл

КОД · JAVA

```
package com.example.bookstore;

import static org.assertj.core.api.Assertions.assertThat;

import java.nio.file.Files;
import java.nio.file.Path;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class BookstoreApplicationTests {
    @Test
    void applicationContextLoadsWithDefaultH2Profile() {
    }

    @Test
    void defaultProfileConfiguresH2SqlLoggingAndConsole() throws Exception {
        String properties =
Files.readString(Path.of("src/main/resources/application.properties"));

        assertThat(properties).contains("spring.datasource.url=jdbc:h2:mem:bookstore");
        assertThat(properties).contains("spring.jpa.show-sql=true");
        assertThat(properties).contains("spring.h2.console.enabled=true");
    }

    @Test
    void mysqlProfileRequiresTheDedicatedDatabasePasswordVariableWithoutFallback() throws
Exception {
        String properties = Files.readString(Path.of("src/main/resources/application-
mysql.properties"));

        assertThat(properties).contains("spring.datasource.url=${BOOKSTORE_MYSQL_URL}");
        assertThat(properties).contains("spring.datasource.username=${BOOKSTORE_MYSQL_USERNAME}");
        assertThat(properties).contains("spring.datasource.password=${BOOKSTORE_DB_PASSWORD}");
```

```
assertThat(properties).doesNotContain("spring.datasource.password=${BOOKSTORE_DB_PASSWORD:");
        assertThat(properties).contains("spring.datasource.driver-class-
name=com.mysql.cj.jdbc.Driver");
    }
}
```

Первый тест ловит ошибки annotations, `mappedBy`, repositories и startup. Второй фиксирует default H2 gate. Третий принадлежит accumulated Optional 09 и статически проверяет MySQL profile, не активируя его и не подключаясь к внешней БД. Для строго чистого Chapter 4 достаточно первых двух методов; листинг оставлен дословно синхронным с готовым cumulative-проектом.

Vendor Bootstrap: не печатать

Minified vendor-файл намеренно не приводится:

- версия: Bootstrap 5.3.3;
- путь: `src/main/resources/static/css/bootstrap.min.css`;
- источник: `https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css`;
- лицензия: MIT, локальная копия `src/main/resources/static/css/bootstrap-LICENSE.txt`;
- SHA-256: `3C8F27E6009CCFD710A905E6DCF12D0EE3C6F2AC7DA05B0572D3E0D12E736FC8`.

Bootstrap — accumulated-оформление, не требование Chapter 4. Не вставляйте 232803 байта minified CSS в отчёт. Checksum: `shasum -a 256` на macOS или `Get-FileHash -Algorithm SHA256` на Windows.

Порядок RED → GREEN

1. Создайте CategoryRepositoryTest: RED на Chapter 3.
2. Добавьте Category/repository и измените Book: GREEN JPA slice.
3. Добавьте падающий MockMvc-тест dropdown, затем categories в Model и `<select>`.
4. Добавьте падающий тест save выбранной category, затем `categoryId`, repository lookup и `setCategory`.
5. Добавьте падающий тест category column, затем измените booklist.
6. Добавьте seed и, если используете accumulated-улучшение, тест повторного запуска.
7. Выполните свежий полный gate.

Запуск и тесты

macOS/Linux:

КОД · BASH

```
cd fullstack-homework-kit/projects/spring-boot/bookstore
chmod +x mvnw
./mvnw clean verify
./mvnw spring-boot:run
```

Windows PowerShell:

КОД · POWERSHELL

```
Set-Location fullstack-homework-kit\projects\spring-boot\bookstore
.\mvnw.cmd clean verify
.\mvnw.cmd spring-boot:run
```

Откройте `http://localhost:8080/booklist`. H2 Console: `http://localhost:8080/h2-console`, JDBC URL из properties, user `sa`, пустой H2 password. Остановка — `Control+C`.

Ручная проверка

1. Clean Code показывает Architecture; Effective Java и Spring in Action — Programming.
2. Add Book содержит Programming, Architecture, Databases.
3. Создайте книгу с Databases; после redirect category видна.
4. Edit сохраняет выбранную category; смените её и сохраните.
5. Удаление книги не удаляет Category.
6. В H2 Console выполните `SELECT * FROM CATEGORY;` и `SELECT TITLE, CATEGORY_ID FROM BOOK;`.
7. POST-тест с неизвестным categoryId возвращает 404.

Типичные ошибки

- В Book нужна `@ManyToOne`; две `@OneToMany` неверны.
- `mappedBy` — имя Java-поля `category`, не таблицы/колонки.
- Изменение только `Category.books` не пишет FK: owner — `Book.category`.
- Без cascade transient Category сохраняют первой.
- Add и edit должны оба класть `categories` в Model.
- `name="categoryId"` и `@RequestParam categoryId` должны совпадать.
- Browser `required` не заменяет server lookup.
- Cascade delete не добавляют без требования.
- MySQL profile не активируют для Chapter 4 tests.

Вопросы и ответы

Где физически хранится связь? В `BOOK.CATEGORY_ID`, потому что `Book.category` — owning side.

Зачем тогда `Category.books`? Обратная коллекция выполняет двустороннее требование и позволяет загрузить книги категории; `mappedBy` не создаёт второй внешний ключ.

Почему Category ищется через repository при save? Так id из недоверенной формы превращается в managed entity, а неизвестное значение получает 404.

Нужно ли отдельное управление категориями? Нет: исходное условие требует entity/repository/seed/dropdown/list, но не CRUD-страницы категорий.

Нужен ли XAMPP? Нет. Обязательный Chapter 4 и его tests используют H2; XAMPP относится только к Optional 09.

Защита

Нарисуйте две таблицы и FK, назовите owner/inverse side и объясните `mappedBy`. Проведите `categoryId` от `<option>` через `@RequestParam` и `CategoryRepository.findById` до `book.setCategory`. Покажите repository/MockMvc tests и отделите обязательные пункты от DTO/404/idempotence/Bootstrap и Optional MySQL.

Чек-лист сдачи

После ручной проверки отправьте именно исходники и тесты, без `target/`, IDE-файлов и локальных credentials. Из корня собственного Bookstore выполните:

КОД · BASH

```
./mvnw clean verify
git status --short
git add pom.xml src/main src/test
git diff --cached --check
git diff --cached
git commit -m "Добавить категории книг и связь one-to-many"
git push
```

Windows PowerShell:

КОД · POWERSHELL

```
.\mvnw.cmd clean verify
git status --short
git add pom.xml src/main src/test
git diff --cached --check
git diff --cached
git commit -m "Добавить категории книг и связь one-to-many"
git push
```

Перед `commit` прочитайте staged diff: там не должно быть `.env`, паролей, `target/` или случайных файлов. После `push` откройте свой GitHub-репозиторий, убедитесь, что последний commit виден и файлы `Category.java`, `CategoryRepository.java`, формы и тесты открываются. Если преподаватель требует отдельную ветку или Pull Request, используйте имя и workflow из GitHub Classroom вместо самостоятельного изменения правил курса.

- ☐ Нарисованы CATEGORY, BOOK и FK `BOOK.CATEGORY_ID`.
- ☐ Объяснены owner `Book.category` и inverse side `Category.books`/`mappedBy`.
- ☐ Category имеет id, name, books, constructors, getters/setters.
- ☐ CategoryRepository расширяет CrudRepository.
- ☐ Initializer сохраняет categories перед books.
- ☐ Add/edit Model содержит categories; dropdown показывает names и отправляет categoryId.
- ☐ Controller разрешает id через repository и вызывает `book.setCategory`.
- ☐ Booklist показывает category name.
- ☐ Repository и MockMvc tests проходят на H2 без MySQL/XAMPP.
- ☐ `clean verify` успешен.

- ☐ Mandatory 04 не смешан с Optional 09 и accumulated-улучшениями.
- ☐ Изменения отправлены в собственный GitHub-репозиторий Bookstore.